

CONCORDIA UNIVERSITY
Gina Cody School of Engineering and Computer Science

SOEN 390
“Software Engineering Team Design Project”
Campus Guide Application
Release 1 Report

Bulat-and-Friends:
Marc-Yves Malchev - 40265238
Joseph Aladas - 40156616
Nektarios Zampetoulakis - 40211948
Ryad Alla - 40227731
Alimurat Dinchdonmez - 40245310
Azmi Abidi - 40248132
Bulat Abdullin - 40264963
Abdullah Taha - 40261146
Alessandro Tiseo - 40262416

9-2-2024

Table of content

1. Introduction.....	3
2. Requirements.....	4
3. Release Planning.....	5
Sprint 1 Summary.....	5
Project Velocity (Sprint 1).....	5
Sprint 2 Summary.....	6
Project Velocity (Sprint 2).....	6
4. Architecture.....	7
4.1. Architecture Overview.....	7
4.2. Domain Model.....	7
4.3. Component Diagram.....	8
4.4. ER Diagram.....	8
4.5. Class Diagram.....	8
4.6. Design Patterns.....	9
4.7. External Libraries.....	9
5. User Interface Design.....	10
5.1. Personas.....	10
5.2. Equity.....	10
5.3. Mock-ups.....	10
6. Testing Plan & Report.....	11
6.1. Unit Testing.....	11
6.2. Test Code Coverage.....	11
6.3. Acceptance Testing.....	12
6.4. System Tests.....	12
6.5. Usability Testing.....	12
7. Defect Tracking Report.....	13
8. Code Review Report.....	13
9. Refactoring Report.....	13
10. Quality Measurements.....	13
11. Professional Ethics & Responsibility.....	13

1. Introduction

The Campus Guide application is designed to help students, faculty members and visitors navigate the university campus efficiently. It provides interactive campus maps, outdoor and indoor navigation, and route planning by integrating services like Google Maps, Concordia Open Data API and Google Calendar to offer real-time directions. Navigating a university campus, especially one with multiple locations and buildings like Concordia's SGW and Loyola campuses, can be challenging. New students, visitors and even faculty members often struggle with finding buildings, classrooms, and important departments. Additionally, indoor navigation is especially difficult, as existing map services do not provide room-to-room directions.

This application aims to solve these problems by offering interactive campus maps with clearly distinguished campus buildings, real-time positioning and additional building information; outdoor and indoor directions with step-by-step navigation, multi-transportation support, and accessibility-friendly routes; classroom navigation that automates directions to upcoming classes using Google Calendar or Concordia Open Data API; points of interest to quickly access nearby restaurants, coffee shops, and essential services; and a smart planner that optimizes task-based routes to minimize walking in the outdoor weather.

The target users of the application include students who need to find classrooms, buildings, and points of interest efficiently; faculty and staff who navigate between buildings and campuses; visitors and newcomers exploring the campus; and individuals with disabilities who require optimized routes with elevators and ramps. By combining outdoor and indoor navigation with real-time data integration, this application significantly enhances campus accessibility and user convenience.

2. Requirements

Backlog

ID	Name	USP	Priority
Sprint 1			
13	Plan your Sprint 2 in Zen Hub by adding some user stories from your backlog (add features that can be completed within 2 weeks)	5	1
12	Think about and document the design of your app (basic domain model, component diagram). Explore external libraries/frameworks that might be useful for your project and discuss their pros and cons	21	2
11	Create UI mock-ups for the user stories you will implement in Sprint 2	13	2
10	As a user, I want the personas to be defined to represent potential users and describe some of the tasks they will perform on the system.	8	2
9	Create your project Backlog in Zen Hub (make a user story for each project requirement)	3	1
8	Set up your GitHub repository and yml scripts for GitHub workflows	13	1
14	Conduct and document your Sprint 1 Retrospective. Meet with your product owner and get feedback for your mock-ups and your backlog	5	3
Total: 68			
Sprint 2			
3	Access information from Concordia's API	8	1
20	Write unit tests for your code	13	2
21	Create UI mock-ups for the user stories you will implement in Sprint 3	13	1
22	Investigate how you can make automated System/UI tests for mobile apps	5	3
23	Manual system testing for the implemented features and report well-documented bug reports on GitHub as issues	2	1
24	Install and run SonarQube for your project. Summarize the issues detected by SonarQube and discuss how and when you will address them	5	2
25	Conduct and document your Sprint 2 Retrospective. Meet with your product owner for acceptance testing and get feedback	8	2
26	Prepare your presentation for the Instructor and practice it with all team members at least once	5	1
27	Submit your Release 1 report on Slack	8	1
Total: 67			

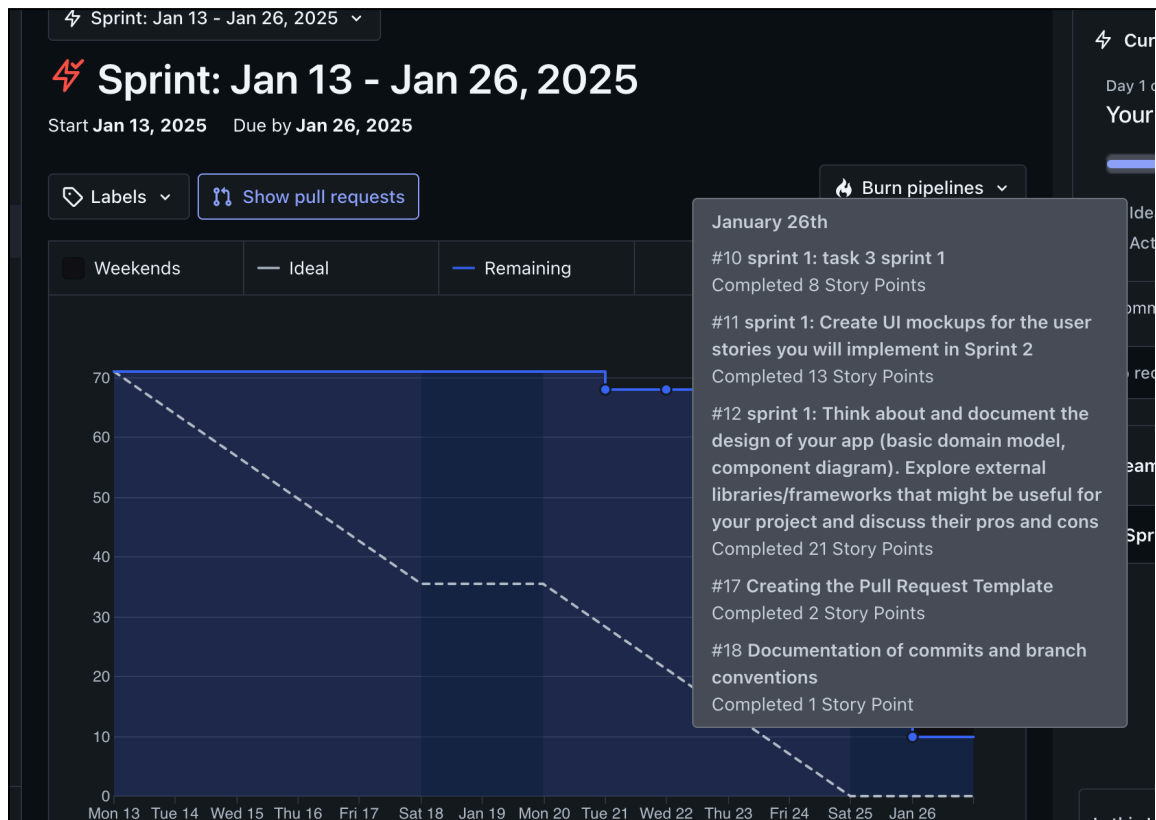
3. Release Planning

Sprint 1 Summary

This sprint focused on laying the groundwork for the project by completing essential setup tasks and delivering the foundational features of the app. The key objectives for this sprint included establishing the development environment, creating initial UI mock-ups and documenting the project requirements.

ID	Explanation	USP	Status
13*	-	5	DONE
12*	-	21	DONE
11*	-	13	DONE
10*	-	8	DONE
9*	-	3	DONE
8*	-	13	DONE
14*	-	5	DONE
Total: 68			

Project Velocity (Sprint 1)

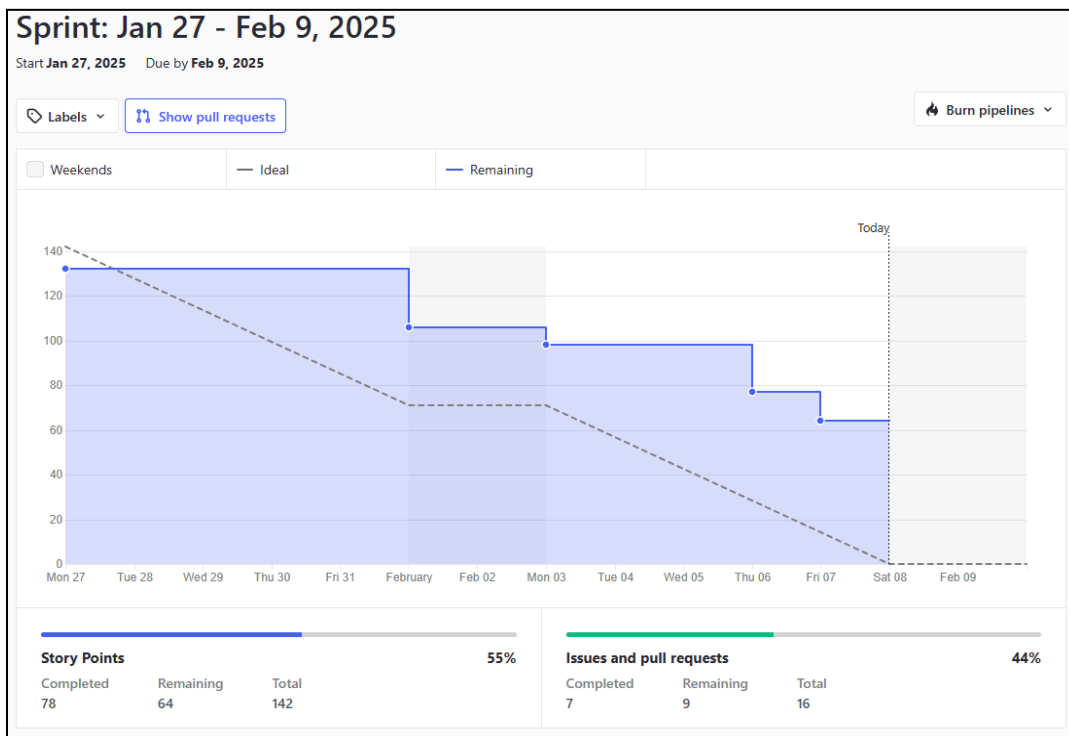


Sprint 2 Summary

This sprint focused more on developing the application, such as acquiring APIs, setting up the DB on Supabase, and developing the source code. At the same time, UI mock-ups were created and SonarQube was introduced to inspect the code. Lastly, extensive documentation of the developed process was issued.

ID	Explanation	USP	Status
3*	-	8	DONE
20*	-	13	DONE
21*	-	13	DONE
22*	-	5	DONE
23*	-	2	DONE
24*	-	5	DONE
25*	-	8	DONE
26*	-	5	DONE
27*	-	8	DONE
Total: 67			

Project Velocity (Sprint 2)



4. Architecture

4.1. Architecture Overview

The Concordia Campus Guide follows the Client-Server Architecture and the MVC Architecture with a Component-Based Design. This architecture is followed to ensure modularity, scalability, and maintainability while providing an efficient and seamless user experience across different devices.

- Client-Server Architecture:

In this architecture, the mobile application, which is the client, interacts with a server to retrieve and update data. This will ensure data in real-time and maintain the simple nature of the client since the heavy computation and database-intensive tasks would occur on the server.

- Component-Based Design:

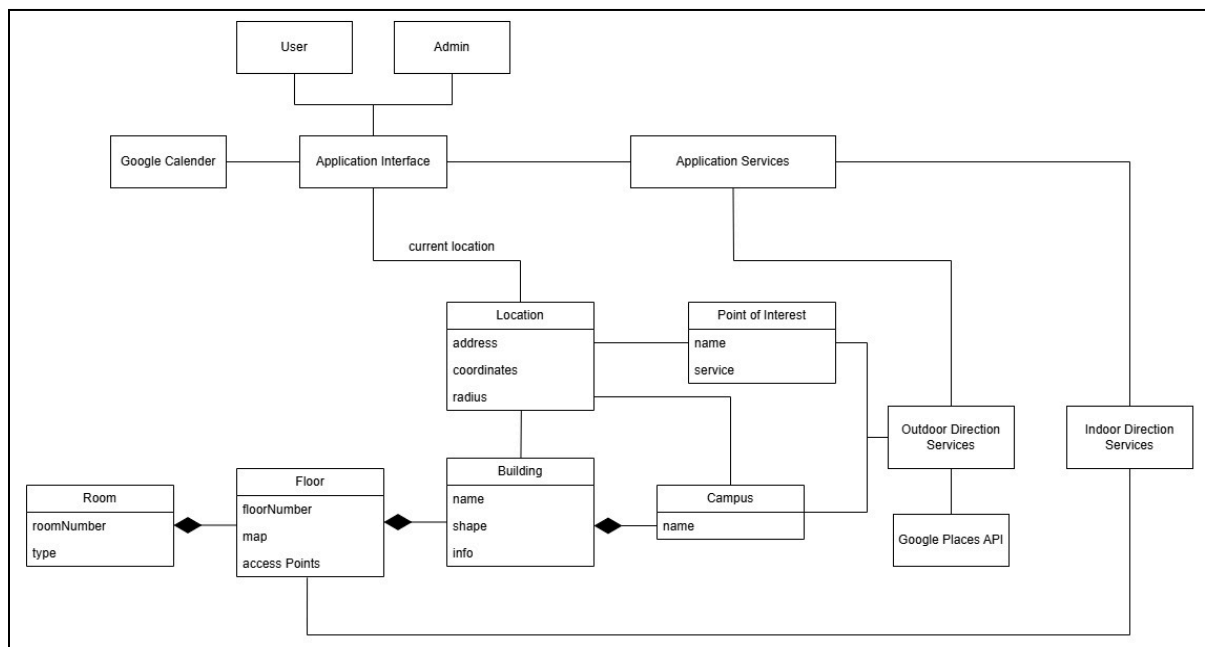
The app will be structured into reusable components for UI, business logic, and data handling. This enhances code reusability (ex: reusable components for buttons and input forms) and simplifies development and testing by isolating components.

- Model-View-Controller (MVC):

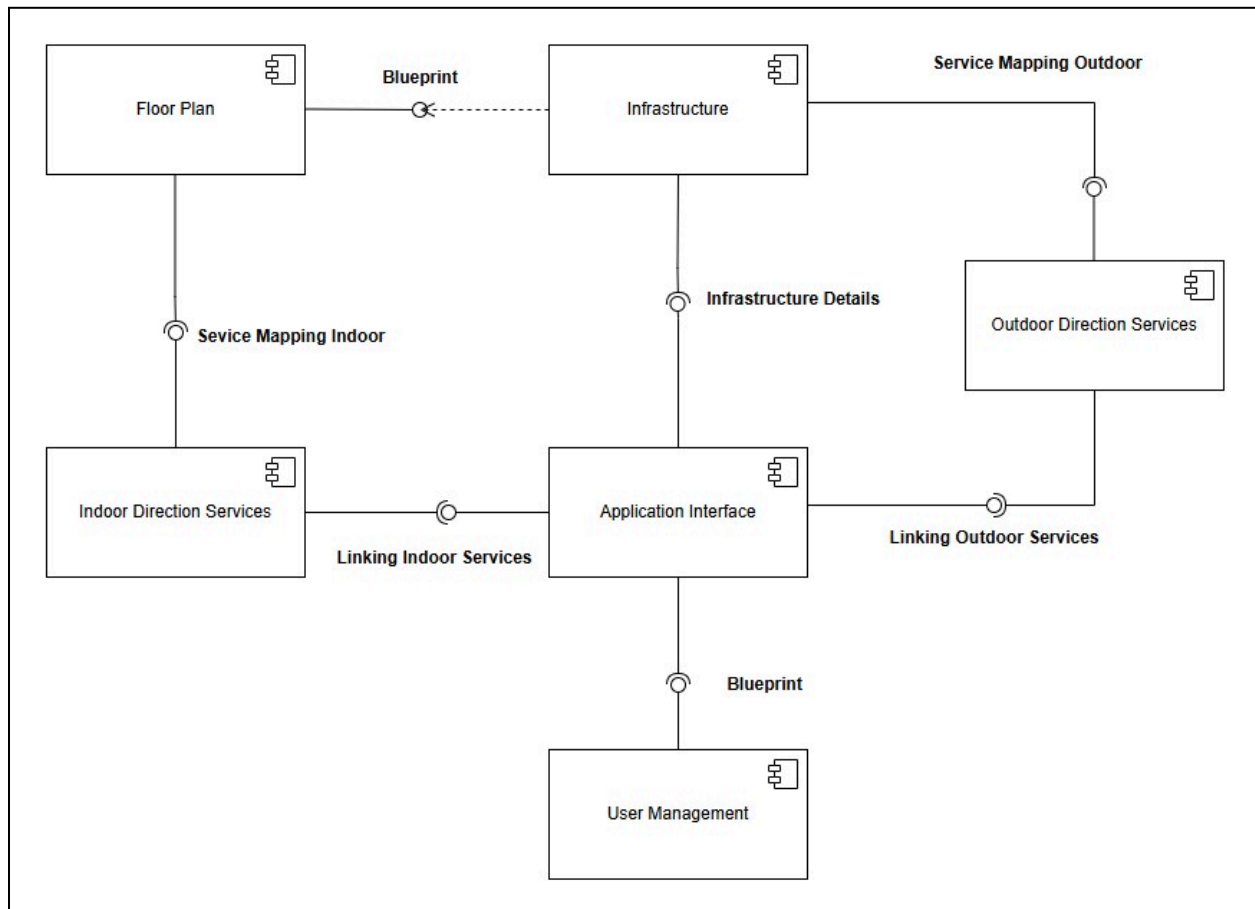
The app loosely follows the MVC pattern to separate concerns:

- Model: Manages data (ex: user information, building details).
- View: Handles the UI (React Native components).
- Controller: Acts as the intermediary, managing logic and API calls.

4.2. Domain Model



4.3. Component Diagram



4.4. ER Diagram

At this point in the project, we are unable to create a complete ER diagram since we only have a table for the buildings. Once more tables and relationships are defined, we will update this section accordingly.

4.5. Class Diagram

At this stage, we have developed a domain model to outline key concepts and their relationships within the system. However, we have not yet created a class diagram because the system's detailed implementation structure, including specific attributes, methods, and dependencies, is still being revised. Once these aspects are clarified, we will create a class diagram to illustrate the concrete design.

4.6. Design Patterns

The application utilizes various Gang of Four (GoF) design patterns, along with framework-specific patterns, to promote modularity, scalability, and maintainability. These patterns ensure the system can evolve efficiently as new requirements are introduced.

- Singleton Pattern

The Singleton Pattern is implemented to ensure a single instance of a class, such as managing application-wide data like user sessions or API clients. For instance, a SessionManager ensures that only one session is maintained for user authentication and API requests throughout the app. This is implemented using React's Context API to manage global state effectively. By coordinating configuration and state management, the pattern ensures consistent behaviour across the app and avoids issues caused by multiple instances.

- Factory Pattern

The Factory Pattern is used to abstract the creation of complex objects, such as navigation flows, based on input parameters. For example, a NavigationFactory generates navigation routes for various scenarios, including outdoor navigation, indoor navigation, or inter-campus navigation. This abstraction simplifies the creation process while hiding the instantiation details, making the system flexible and maintainable. It also makes it easy to introduce new navigation types, such as accessibility-specific routes, by extending the factory to support additional modes without modifying existing code.

- Strategy Pattern

The Strategy Pattern is employed to define a family of algorithms, encapsulate them, and make them interchangeable, such as in selecting transportation modes for outdoor navigation. Each transportation mode, like walking, driving or shuttle, is encapsulated as a separate strategy, allowing seamless switching based on user preferences or environmental factors. This modular design promotes clean and flexible code, making it easier to add new strategies, such as biking routes or eco-friendly paths, without altering the existing logic.

- Framework-Specific Patterns

The Component Pattern, a fundamental aspect of React Native, is used to encapsulate UI elements into reusable, self-contained components. Examples include buttons, models, and map markers, which are designed to be modular and easy to test. This approach ensures that the codebase remains clean and adheres to the DRY (Don't Repeat Yourself) principle, enabling rapid development and straightforward scaling of the app's interface as new features are introduced.

4.7. External Libraries

For the Campus Guide application, we decided to use Google Maps API because it is reliable, easy to integrate, and well-documented. It offers both outdoor and indoor navigation, supports multiple modes of transportation, and integrates flawlessly with real-time location services. This choice was made based on our team's familiarity with Google APIs, their frequent updates, and their multiple online resources.

5. User Interface Design

5.1. Personas

All the user personas defined can be found in the [repo](#) in the documents/sprint1 folder

5.2. Equity

The Concordia Campus Guide supports accessibility and equity through features that accommodate users with disabilities, illiterate users, and other underrepresented groups. To assist them, the application offers high-contrast visuals for colour-blind users, compatibility with screen readers for blind users, visual and text-based alternatives for deaf users, and support for voice commands for those with hand mobility difficulties. For illiterate users, the app uses intuitive icons, images, and audio-based instructions to simplify navigation. Underrepresented groups benefit from offline functionality, light versions for low-end devices, and culturally sensitive content. Conformance with accessibility standards such as WCAG and regular usability testing ensures the app remains accessible and equitable for all.

5.3. Mock-ups

All UI Mock-ups can be found on [FIGMA](#).

6. Testing Plan & Report

6.1. Unit Testing

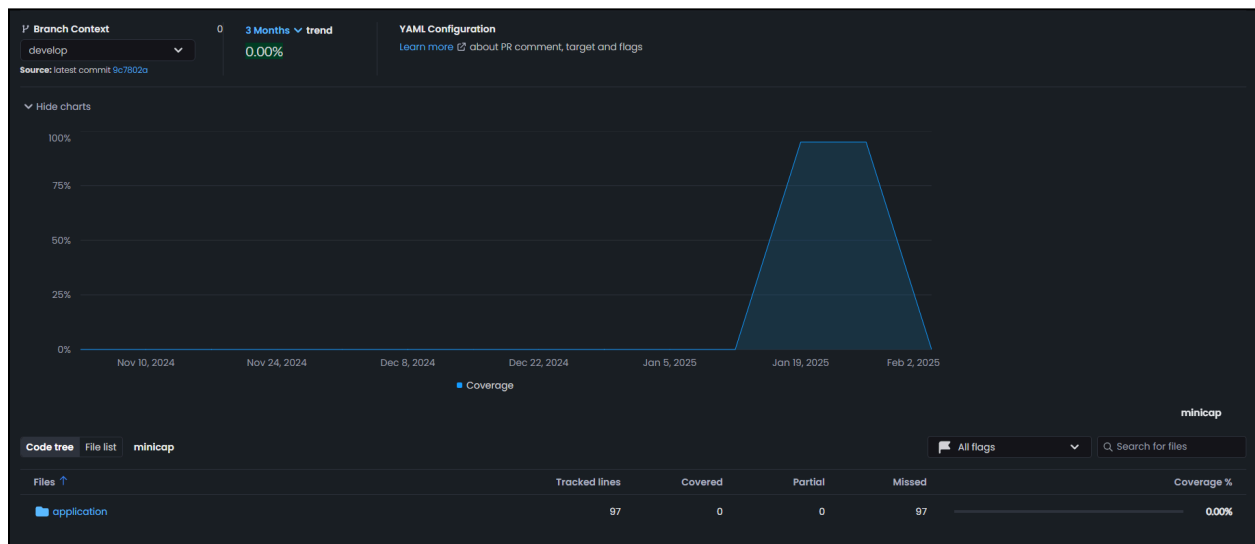
We haven't implemented many unit tests yet but plan to use Jest for testing core functions and components. UI interactions cannot be tested with unit tests and will instead be handled through End-to-End (E2E) testing.

```
Run tests

1  ▶ Run npm run test -- --coverage
4
5  > minicap_concordia_campus_guide_app@1.0.0 test
6  > jest --ci --coverage
7
8  PASS components/__tests__/ThemedText-test.tsx
9    ✓ renders correctly (420 ms)
10
11  -----|-----|-----|-----|-----|-----
12  File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
13  -----|-----|-----|-----|-----|-----
14  All files  |    90.9 |    53.33 |    100 |    90.9 |
15  components |    100 |    54.54 |    100 |    100 |
16  ThemedText.tsx |    100 |    54.54 |    100 |    100 | 24-28
17  constants  |    100 |    100 |    100 |    100 |
18  Colors.ts   |    100 |    100 |    100 |    100 |
19  hooks       |     80 |     50 |    100 |     80 |
20  useColorScheme.ts |     0 |     0 |     0 |     0 |
21  useThemeColor.ts |     80 |     50 |    100 |     80 | 17
22  -----|-----|-----|-----|-----|-----
23  Test Suites: 1 passed, 1 total
24  Tests:       1 passed, 1 total
25  Snapshots:   1 passed, 1 total
26  Time:        2.11 s
27  Ran all test suites.
```

6.2. Test Code Coverage

Our CodeCov coverage dropped from 95% to 0%, likely due to a CI/CD pipeline issue or yaml file configuration. We will investigate and fix the issue to restore accurate coverage tracking.



6.3. Acceptance Testing

During Sprint 2, we were unable to meet with our product owner, so we couldn't complete the acceptance testing process. We had four acceptance tests planned: ensuring that Concordia buildings appear in a different colour, allowing users to click on a building to view its information, enabling users to switch between both campuses and displaying the building the user is currently in. These tests were not officially reviewed, but we will address them in the next sprint.

6.4. System Tests

We will use Maestro for system testing to automate interactions and validate feature functionality. Maestro will simulate user actions, such as navigating between campuses and viewing building information. Each test will follow specific steps, like clicking on a specific campus map or a specific building and seeing its information. The test executions will be recorded and added to our repository for reference, ensuring that all features work as expected.

```
Running on emulator-5554
```

```
> Flow: campus_navigation_test
```

```
+ Clear state of com.bu_abd.minicap_concordia_campus_guide_app
+ Launch app "com.bu_abd.minicap_concordia_campus_guide_app"
+ Assert that "Link to SGW" is visible
+ Assert that "Link to LOY" is visible
+ Tap on "Link to SGW"
+ Assert that "SGW map" is visible
+ Press back
+ Tap on "Link to LOY"
+ Assert that "Loyola map" is visible
+ Swipe from (50%, 70%) to (50%, 30%) in 1000 ms
+ Swipe from (50%, 30%) to (50%, 70%) in 1000 ms
+ Press back
```

6.5. Usability Testing

We have not yet done any usability testing. However, we researched Google Lighthouse and could potentially use it to assess user experience, including factors like page load time and accessibility. We will define appropriate metrics, such as task completion rate and user satisfaction, to then analyze the results and implement any necessary improvements in future releases.

7. Defect Tracking Report

- <https://github.com/AlimuratDinch/minicap/issues/55>

8. Code Review Report

- During the code review of [PR #35](#), we discussed colour-coding buildings in the UI mock-ups to differentiate their purposes (academic vs. administrative). It was clarified that the original orange and grey colours were based on Google Maps. The pull request was also linked to an issue for automatic closure once merged.

- [PR #32](#) added a document explaining the chosen frameworks, libraries, and APIs for the Campus Guide Mobile App. It was suggested to link it to issue GH-12, which was implemented.

- [PR #34](#) aimed to add the component diagram to the documentation folder for Sprint 1. Reviewers reminded the author to check the completed tasks in the PR checklist for better tracking.

- [PR #33](#) aimed to add a document containing a PDF of 4 user personas, with no code changes. Reviewers approved the content but suggested improving commit messages by adding tags (ex: feat, docs, test, ...) and using lowercase letters.

- [PR #30](#) aimed to create a CI pipeline. The initial YML file and test coverage were confirmed to work. Reviewers approved the changes and added an image of the results obtained.

9. Refactoring Report

We refactored the project repository by reorganizing its structure. We divided the repository into two main folders: one for everything related to the application itself and another for all project-related documentation. Inside the documentation folder, we created subfolders for each Sprint, containing related documents, as well as a folder for the releases. This refactoring was done to improve the organization of the project, making it easier to navigate and manage both the application and the documentation. - <https://github.com/AlimuratDinch/minicap/pull/40>

10. Quality Measurements

The SonarQube report can be found [here](#).

11. Professional Ethics & Responsibility

The Concordia Campus Guide is generally designed to have a positive impact by improving campus navigation and accessibility. However, ethical concerns could appear in areas such as data privacy and reliance on technology. The app collects user data, such as location, which raises concerns about privacy and security. To address this, the app follows strict data protection practices. Reliance on technology can pose challenges if users overly depend on the app and face disruptions during technical issues. To address this, the app includes fail-safe features, such as printable directions or maps of the different buildings on both campuses, ensuring it supports users even in challenging scenarios. These measures demonstrate a commitment to ethical design and minimizing potential negative impacts.